

UNIVERSITY OF TEXAS AT ARLINGTON

System On Chip Design (CSE5392-056)

FALL 2025

**Design of a I2C IP Core and
Linux Device Drivers**

Report by,

Rezwana Karim Roza (1001919948)

1. Introduction.....	3
2. System Architecture.....	3
3. Hardware Functional Description (PL).....	4
3.1 Clock Divider.....	5
3.2 Transmit and Receive FIFOs.....	5
i) Transmit FIFO (TX FIFO).....	5
ii) Receive FIFO (RX FIFO).....	6
3.3 Memory-Mapped Register Interface.....	6
3.4 Finite State Machine.....	7
4. Software Implementation.....	8
4.1 kernel base driver (kernbase).....	9
4.2 Kernel Expander Driver (kernex).....	9
4.3 user-space (user).....	10
5. I2C IP Execution/ User Manual.....	11
5.1 Build and Basic User Test (general).....	11
5.2 Load Kernel Modules (kernbase and kernex).....	12
5.3 Kernel Manual.....	12
5.4 User Mode.....	12
6. Testing and Results.....	13
7. TI Reference Waveform.....	13
Conclusion.....	14
Reference.....	15

1. Introduction

The main objective of this project is to design and implement a complete hardware - software I2C master subsystem on the Blackboard FPGA platform, which is based on a Xilinx Zynq device containing both an FPGA fabric and an ARM-based processor subsystem. The hardware component consists of a custom Verilog I2C IP core that communicates with standard 7-bit I2C peripherals and exposes a memory-mapped AXI-Lite interface to the processor. The software component includes both a user-space testing application and Linux kernel drivers that provide structured access to the I2C core through the sysfs virtual file system. The design bridges the programmable logic (PL) and processing system (PS) portions of the Blackboard by exposing the custom I2C hardware module to the PS through a memory-mapped AXI-Lite interface. This interface enables Linux running on the PS to configure the IP core, initiate transactions, and exchange data with external I2C peripherals.

Together, these components demonstrate a full hardware–software co-design workflow in which a custom FPGA peripheral is seamlessly integrated into a Linux-based embedded environment. The work spans synchronous digital design, register-mapped hardware integration, kernel-space driver development, and user-space control. The system is validated using an MCP23008 I/O expander, enabling reliable configuration and manipulation of GPIO direction, pull-up resistors, and pin-level data through the implemented I2C interface.

2. System Architecture

This project is implemented on the Blackboard platform, which uses a Xilinx Zynq device that combines a Processing System (PS) and Programmable Logic (PL) on a single chip. In this design, the custom I2C master IP core is implemented in the PL, and Linux runs on the PS. The I2C core is connected to the PS through a 32-byte AXI4-Lite aperture starting at physical address `AXI4_LITE_BASE + I2C_BASE_OFFSET` (0x43C00000 in this design). Within this aperture, five 32-bit registers (ADDRESS, REGISTER, DATA, STATUS, and CONTROL) are defined using the offsets in `i2c_regs.h`

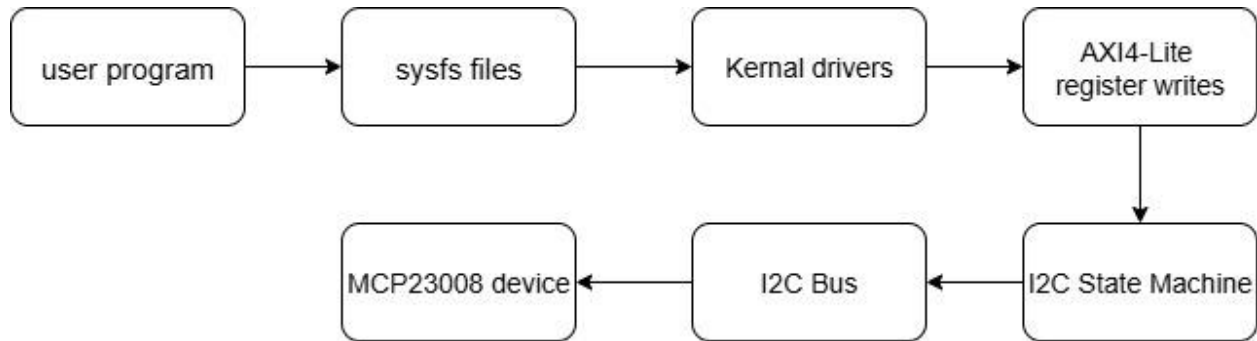
```
#define AXI4_LITE_BASE      0x43C00000
#define I2C_BASE_OFFSET    0x00000000
```

On the PL side, the I2C IP core drives the open-drain SDA and SCL lines that are routed to a PMOD connector and connected externally to an MCP23008 I/O expander. The core contains a clock-divider, a transmit FIFO, a receive FIFO, and a byte-level state machine that implements START and STOP conditions, address and data phases, ACK/NACK checking, and repeated-start. The STATUS and CONTROL registers expose FIFO status, ACK error and busy flags, and control bits such as read/write mode, byte count, use-register, repeated-start, and start.

On the PS side, the AXI-mapped register space is accessed through two software layers. The first layer is a low-level kernel module implemented in the **kernbase** directory (`i2c_base.c`). This module uses `ioremap()` to map the AXI4-Lite register aperture into kernel virtual memory and exposes each hardware control field as a sysfs attribute under `/sys/kernel/i2c/`, including mode, byte_count, register, address, use_repeated_start, tx_data, rx_data, and start. The second layer is an abstraction driver located in the

kernex directory (`i2c_expander.c`), which builds on the base driver and exposes each MCP23008 pin through a structured group of sysfs files at `/sys/kernel/i2c_expander/<any MCP23008 PIN>/{dir,pullup,data}`. This transforms the I2C I/O expander into a virtual GPIO device accessible directly from Linux without any elevated privileges.

User-space applications form the top layer of the system. The `i2c_ip.c` library reads and writes the sysfs attributes to control the I2C core and the expander, and the `i2c` command-line program provides an interface for configuring transfers, reading and writing MCP23008 registers, and running higher-level tests such as the stop-go LED/button demo. From the application's perspective, the interaction path is:



3. Hardware Functional Description (PL)

The programmable logic (PL) part of the system contains a complete I2C bus implemented in SystemVerilog. The design integrates several key components such as an internal clock divider, transmit and receive FIFOs, a multi-stage I2C state machine, and an AXI4-Lite register interface, working together to generate the full I2C protocol on the SDA and SCL lines. Once software provides the configuration parameters and asserts the START bit, the hardware autonomously performs all I2C signaling without requiring CPU involvement.

3.1 Clock Divider

The module `i2c_clock.sv` divides the system clock to produce a 200 kHz clock-enable pulse, which the state machine uses to time SDA and SCL transitions. Every I2C bit requires two enable pulses: one to update SDA and one to toggle SCL. This clock-enable signal can be connected to a physical pin through the TEST_OUT bit in the CONTROL register, allowing external verification using a ILA or oscilloscope.

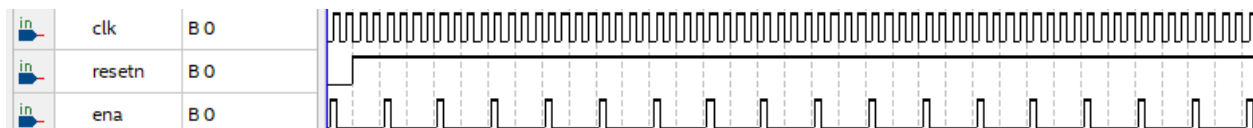


Fig 1: waveform of Test_out

3.2 Transmit and Receive FIFOs

Two independent FIFOs buffer data between the software and the state machine.

i) Transmit FIFO (TX FIFO)

- Receives bytes written by software via the DATA register
- Stores up to 15 bytes
- Provides status flags (empty, full, overflow) reflected in the STATUS register
- Reset clears all stored bytes, full/empty flags, overflow flags, and index counters
- It supplies one byte at a time to the FSM during transmission

```
fifo15x8 fifo_i2c_tx
(
    .clk(axi_clk),
    .reset(~axi_resetrn | manual_reset),

    .wr_request(tx_wr_request),
    .wr_data(tx_wr_data),
    .wr_index(tx_wr_index),

    .rd_request(tx_rd_request),
    .rd_data(tx_rd_data),
    .rd_index(tx_rd_index),

    .empty(tx_empty),
    .full(tx_full),

    .clear_overflow_request(fifo_tx_clear_req),
    .overflow(tx_overflow)
);
```

ii) Receive FIFO (RX FIFO)

- It accepts bytes received from the I2C bus
- It captures up to 15 bytes
- Provides corresponding status flags
- It allows the software to read received bytes through the DATA register

```

fifo15x8 fifo_i2c_rx
(
    .clk(axi_clk),
    .reset(~axi_resetsn | manual_reset),

    .wr_request(rx_wr_request),
    .wr_data(rx_wr_data),
    .wr_index(rx_wr_index),

    .rd_request(rx_rd_request),
    .rd_data(rx_rd_data),
    .rd_index(rx_rd_index),

    .empty(rx_empty),
    .full(rx_full),

    .clear_overflow_request(fifo_rx_clear_req),
    .overflow(rx_overflow)
);

```

3.3 Memory-Mapped Register Interface

The connection between the Processing System and the I2C hardware in the Programmable Logic is implemented using a standard AXI4-Lite slave interface. This interface is responsible for transferring configuration and status information between Linux and the custom I2C core using simple 32-bit memory-mapped read and write operations. The file `i2c_slave_lite_v1_0_AXI_Interface.v` wraps the I2C core with an AXI-Lite slave interface. Five 32-bit registers are exposed to the PS.

- ADDRESS : 7-bit I2C slave address
- REGISTER : internal register index for devices like MCP23008
- DATA : write pushes to TX FIFO; read pops from RX FIFO
- STATUS : FIFO status, ACK error, busy flag, debug bits
- CONTROL : RW mode, byte count, use-register flag, repeated-start, test-out, and START bit

Register	Reg Offset	Bits	Field	Function
ADDRESS (R/W)	0	6:0		7-bit address of the device
REGISTER	4	7:0		8-bit register number
DATA (R/W)	8	7:0		Write: Write data to the TX FIFO Read: Reads data from the RX FIFO
STATUS (R/W1C)	12	0	RXFO	RX FIFO overflow (write 1 to clear)
		1	RXFF	RX FIFO full (auto clears when data is read)
		2	RXFE	RX FIFO empty (auto clears when data arrives)
		3	TXFO	TX FIFO overflow (write 1 to clear)
		4	TXFF	TX FIFO full (auto clears when data is sent)
		5	TXFE	TX FIFO empty (auto clears when data is written)
		6	ACK_ERROR	An ACK error occurred (write 1 to clear)
		7	BUSY	The module is busy transmitting or receiving
		31:8	DEBUG_IN	Debug interface (often used to peek at FSM)
CONTROL (R/W): reset 0	16	0	R/~W	Direction of data on bus (read=1, write=0)
		4:1	BYTE_COUNT	Number of bytes to be written or read
		5	USE_REGISTER	When reading and writing data from devices using internal registers, the register number should be sent after the address is sent.
		6	USE_REPEATED_START	1 = use repeated start conditional on read, 0 = use stop followed by start condition on reads
		7	START	Write: 1 = start bus transaction, 0 = no effect Read: 0
		8	TEST_OUT	1 = enable test output, 0 = disable test output
		23:9	-	<i>reserved</i>
		31:24	DEBUG_OUT	Debug interface (can be used to drive GPO or LED ports)

3.4 Finite State Machine

The I2C master behavior is implemented in a FSM. Once software writes configuration values into the ADDRESS, REGISTER, DATA, and CONTROL registers and asserts the START bit, the state machine drives every aspect of the I2C protocol in hardware. The FSM generates START/STOP conditions, transmits the address and optional register byte, checks ACK responses, and then it either sends data from the TX FIFO or receives data into the RX FIFO depending on the R/W bit. For read transactions, the FSM supports repeated-start sequences and automatically sends ACK/NACK during multi-byte reads. Throughout the transaction, the BUSY flag, ACK error, and debug state field in the STATUS register reflect the internal FSM state. When the STOP condition is generated, the FSM returns to IDLE and clears BUSY.

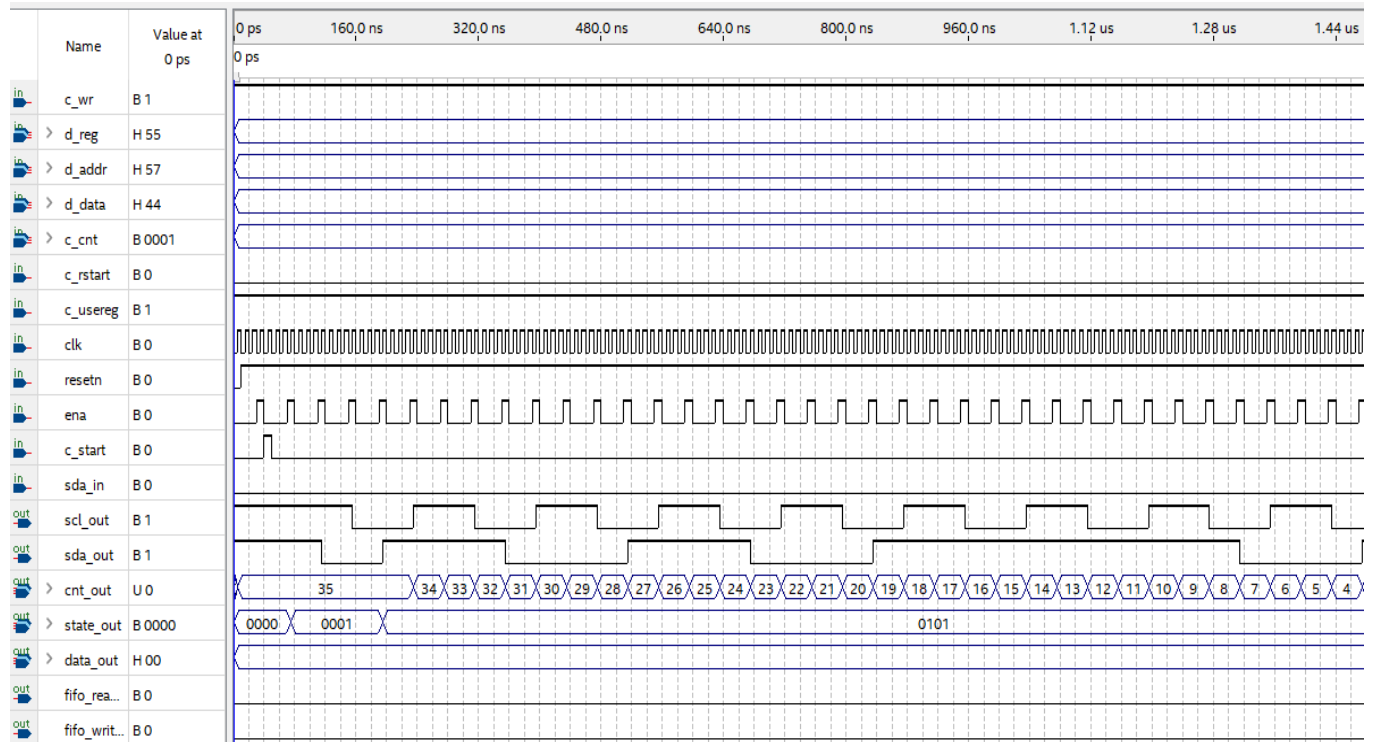


Fig 2: FSM waveform

4. Software Implementation

The software implementation for this project consists of a layered architecture that connects Linux running on the Processing System to the I2C hardware peripheral implemented in the Programmable Logic. So it connects the PL side with the PS side with AXI- Lite. At the lowest level, the kernel base driver maps the AXI-Lite register space of the I2C IP core into kernel memory using `ioremap()` and exposes each hardware control field of mode, byte count, register index, device address, repeated-start, transmit data, receive data, and the start command, through a collection of sysfs files under `/sys/kernel/i2c/`. This allows user space to interact with the hardware through simple file reads and writes rather than raw register access or elevated privileges. Building on this, the kernel expander driver provides a higher-level interface for the MCP23008 I/O expander. It creates a virtual gpio hierarchy under `/sys/kernel/i2c_expander/`, where each pin (0–7) appears as a set of attributes for direction, pull-up configuration, and pin data. Internally the expander driver basically translates these operations into the appropriate I2C transactions by calling into the base driver which handles MCP23008 register operations such as IODIR, GPPU, GPIO, and OLAT. At the top of the implementation, a user-space command-line program interacts with both drivers through sysfs. This application provides an interface for configuring the I2C core that sends and receives bytes, inspecting status flags, and controlling the MCP23008 pins. Together, these three layers of base driver, expander driver, and user-space form a complete software stack that enables the communication between Linux and the I2C hardware.

4.1 kernel base driver (kernbase)

The kernel base driver is located in my kernbase folder and serves as the main link between Linux and the I2C hardware. This driver uses ioremap() to map the AXI-Lite registers of the I2C IP core into kernel memory so the processor can read and write them safely. It then exposes each hardware setting such as the I2C mode, byte count, register number, device address, repeated-start setting, transmit data, receive data, and the START command through simple sysfs files. So the user programs can control the hardware just by reading and writing to files instead of using sudo or any elevated privileges.

```
//-----
// Initialization and Exit
//-----

static int __init initialize_module(void)
{
    int result;

    printk(KERN_INFO "I2C: starting\n");

    kobj = kobject_create_and_add("i2c", kernel_kobj);
    if (!kobj)
    {
        printk(KERN_ALERT "I2C: failed to create and add kobj\n");
        return -ENOENT;
    }

    result = sysfs_create_file(kobj, &mode_attr.attr);
    if (result) return result;
    result = sysfs_create_file(kobj, &byte_count_attr.attr);
    if (result) return result;
    result = sysfs_create_file(kobj, &register_attr.attr);
    if (result) return result;
    result = sysfs_create_file(kobj, &address_attr.attr);
    if (result) return result;
    result = sysfs_create_file(kobj, &use_repeated_start_attr.attr);
    if (result) return result;
    result = sysfs_create_file(kobj, &start_attr.attr);
    if (result) return result;
    result = sysfs_create_file(kobj, &tx_data_attr.attr);
    if (result) return result;
    result = sysfs_create_file(kobj, &rx_data_attr.attr);
    if (result) return result;

    i2c = (unsigned int*)ioremap(AXI4_LITE_BASE + I2C_BASE_OFFSET, SPAN_IN_BYTES);
    if (i2c == NULL) return -ENODEV;
    printk(KERN_INFO "I2C: initialized\n");
    return 0;
}
```

4.2 Kernel Expander Driver (kernex)

The kernel expander driver is located in my kernex folder. This driver makes the MCP23008 I/O expander much easier to use. Instead of having to manually send I2C commands for every register change, the driver gives each MCP23008 pin its own set of files under /sys/kernel/i2c_expander/. Every pin (0 - 7) has files like dir, pullup, and data, and the pin can be controlled by writing a value to these files.

```

//-----
// Initialization and Exit
//-----

static int __init initialize_module(void)
{
    int result;

    printk(KERN_INFO "I2C Expander: starting\n");

    kobj = kobject_create_and_add("i2c_expander", kernel_kobj);
    if (!kobj)
    {
        printk(KERN_ALERT "I2C Expander: failed to create and add kobj\n");
        return -ENOENT;
    }

    result = sysfs_create_group(kobj, &group0); if (result != 0) return result;
    result = sysfs_create_group(kobj, &group1); if (result != 0) return result;
    result = sysfs_create_group(kobj, &group2); if (result != 0) return result;
    result = sysfs_create_group(kobj, &group3); if (result != 0) return result;
    result = sysfs_create_group(kobj, &group4); if (result != 0) return result;
    result = sysfs_create_group(kobj, &group5); if (result != 0) return result;
    result = sysfs_create_group(kobj, &group6); if (result != 0) return result;
    result = sysfs_create_group(kobj, &group7); if (result != 0) return result;

    i2c = (unsigned int*)ioremap(AXI4_LITE_BASE + I2C_BASE_OFFSET, SPAN_IN_BYTES);
    if (i2c == NULL) return -ENODEV;
    printk(KERN_INFO "I2C Expander: initialized\n");
    return 0;
}

```

4.3 user-space (user)

The user-space is the final layer of the software that runs from the Linux terminal to test and control the I2C hardware. This program uses the sysfs files created by the kernel drivers, so it doesn't need any special permissions or direct access to hardware registers. Instead of dealing with low-level I2C protocol steps, the user types simple commands like setting the mode, the device address, selecting a register, writing a byte, or reading data. The program also includes commands for working with the MCP23008 expander like changing a pin's direction. Each command is translated into a read or write to the appropriate sysfs file, which then triggers the kernel drivers to perform the actual I2C transaction. This makes it easy to test individual features, debug the hardware, and verify that the entire system implementation is working correctly.

```

else if (argc == 2 && (strcmp(argv[1], "-h") == 0 || strcmp(argv[1], "--help") == 0))
{
    printf("    usage:\n");
    printf("    ./i2c mode                get the mode value\n");
    printf("    ./i2c mode {value}       set the mode value\n");
    printf("    ./i2c cnt                get the byte count value\n");
    printf("    ./i2c cnt {value}       set the byte count value\n");
    printf("    ./i2c reg                get the register value\n");
    printf("    ./i2c reg {value}       set the register value\n");
    printf("    ./i2c addr               get the address value\n");
    printf("    ./i2c addr {value}      set the address value\n");
    printf("    ./i2c rstart             get the use_repeated_start value\n");
    printf("    ./i2c rstart {value}    set the use_repeated_start value\n");
    printf("    ./i2c start              start the transaction\n");
    printf("    ./i2c data               get the rx_data value\n");
    printf("    ./i2c data {value}      set the tx_data value\n");
    printf("\n");
    printf("    ./i2c pin {pin} dir      read pin direction value\n");
    printf("    ./i2c pin {pin} dir {value} write pin direction value\n");
    printf("    ./i2c pin {pin} pull     read pin pull-up value\n");
    printf("    ./i2c pin {pin} pull {value} write pin pull-up value\n");
    printf("    ./i2c pin {pin} data     read pin data\n");
    printf("    ./i2c pin {pin} data {value} write pin data\n");
    printf("\n");
    printf("    ./i2c stopgo1           test stop go 1\n");
    printf("    ./i2c stopgo2           test stop go 2\n");
    printf("    where {pin} is 0-7\n");
}

```

5. I2C IP Execution/ User Manual

5.1 Build and Basic User Test (general)

1. **Move to the general directory**
cd ~/i2c/general
2. **Build the program**
make -B
3. **Check for available commands**
./i2c.elf -h
4. **Initialize the I2C IP**
sudo ./i2c.elf init
5. **Reset I2C IP**
sudo ./i2c.elf reset
6. **I2C IP Status**
sudo ./i2c.elf status

5.2 Load Kernel Modules (kernbase and kernex)

1. **Load the kernel base driver**
`sudo insmod i2c_base.ko`
2. **Load the I/O expander driver**
`sudo insmod i2c_expander.ko`
3. **Verify that the sysfs entries are active**
`ls /sys/kernel/i2c`
`ls /sys/kernel/i2c_expander`
4. **Unload the base driver**
`sudo rmmod i2c_base`

5.3 Kernel Manual

1. **Write and read I2C core**
`echo 1 >> /sys/kernel/i2c/mode`
`echo 1 >> /sys/kernel/i2c/start`
`echo 1 >> /sys/kernel/i2c/tx_data`

`cat /sys/kernel/i2c/mode`
`cat /sys/kernel/i2c/rx_data`
`cat /sys/kernel/i2c/start`
2. **Write and read MCP23008 expander pin**
`ls /sys/kernel/i2c_expander`
`echo 1 >> /sys/kernel/i2c_expander/0/data // set pin 0 high`
`cat /sys/kernel/i2c_expander/0/pullup // read pull-up setting for pin 0`
`cat /sys/kernel/i2c_expander/0/data // read pin 0 value`

5.4 User Mode

1. **Stop go Commands for both Kernels**
`~/i2c/user/i2c.elf stopgo1`
`~/i2c/user/i2c.elf stopgo2`

6. Testing and Results

To verify the functionality of the I2C IP core, a series of tests were performed involving the MCP23008 I/O expander and the provided stop/go demo. These tests validated correct operation of the PL hardware, the AXI-Lite interface, the kernel drivers, and the user-space testing tools. The results confirmed that the custom I2C core is able to successfully transmit and receive data, handle register addressing sequences, support repeated-start conditions, and correctly interface with an external I2C peripheral.



Fig 3 : waveform Stopgo (yellow - scl, green - sda, and blue - test_out)

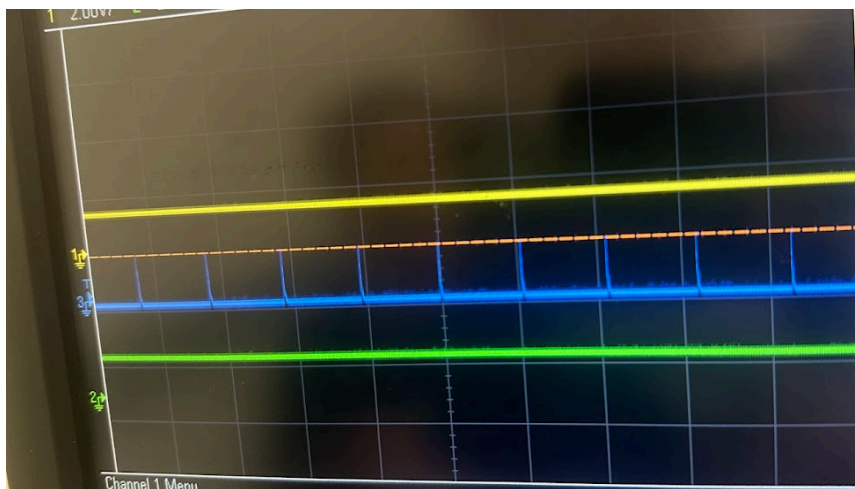


Fig 4: Test_out Clock (blue)

7. TI Reference Waveform

To confirm that the custom I2C implementation follows the I2C timing and transaction format, the read sequence generated by our hardware was compared with the Texas Instruments (TI) reference waveform for rd and wr. The TI diagrams illustrate the required sequence of events, including the initial START

condition, device address with $R/W = 0$, register address write, ACK sampling, repeated START, device address with $R/W = 1$, data reception with ACK/NACK, and the final STOP condition.

When observing the SDA/SCL signals using a logic analyzer and scope, the waveform produced by the FSM matched the TI reference exactly. START and STOP conditions were properly formed, ACK bits were sampled at the correct clock edges, the repeated START timing was followed, and the final NACK and STOP were generated as specified. This confirms that the hardware design is protocol compliant and correctly implements the I2C write and read cycle.

Write to One Register in a Device

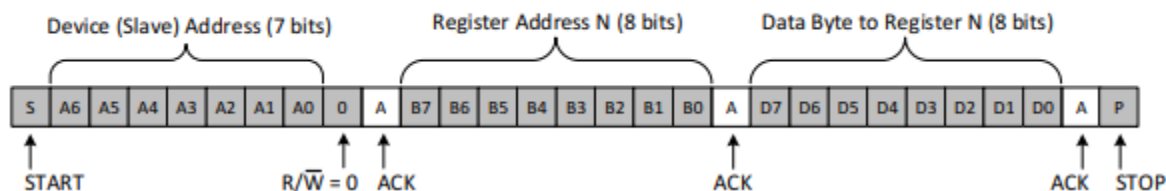


Fig 5: write sequence of TI I2C reference manual

Read From One Register in a Device

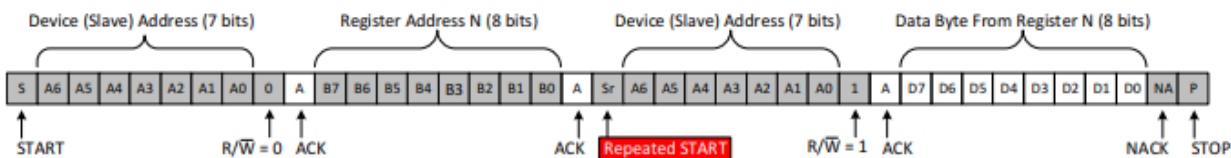


Fig 6: complex read sequence of TI I2C reference manual

Conclusion

This project brought together the full path from hardware design in the programmable logic to software control in the Linux operating system. By building a custom I2C master core in Verilog, connecting it to the ARM processor through an AXI-Lite interface, and writing both kernel drivers and a user-space testing tool, I was able to create a complete I2C communication system. The hardware correctly handled all parts of the I2C protocol START and STOP conditions, ACK/NACK detection, register addressing, repeated-start sequences, and multiple byte transfers. On the software side, the layered design made the system easy to test and use the kernbase driver exposed the hardware cleanly through sysfs, the expander driver made the MCP23008 behave like a simple GPIO device, and the Command line program provided an intuitive way to run tests.

Reference

I2C Bus Specification-User's Guide. Texas Instruments, 2006,
<https://www.ti.com/lit/an/slva704/slva704.pdf>